

Architecting the DataSimple AI Tutor: A Serverless Retrieval-Augmented Generation and Multi-Agent Pipeline

The Vision of an Accessible Data Science Educator

The transition from a traditional computer science mentality to a modern, data-driven workflow represents a significant psychological and technical hurdle for many learners entering the analytics space. Historically, programming has been taught through a trial-by-fire methodology, focusing heavily on raw functions, loops, and abstract algorithms in isolation before ever introducing practical applications. The curriculum philosophy at DataSimple.education directly challenges this antiquated paradigm. The platform operates on the fundamental belief that learning data science with Python can be highly accessible, provided the student is guided through the difficult mental transition to a "coder mindset".

By prioritizing practical, higher-level tools like Pandas, Scikit-learn, and TensorFlow from the very beginning, the educational framework treats Python not merely as an abstract programming language, but as an interactive utility, just like a computer mouse. Just as a user does not need to understand the underlying hardware schematics of a physical mouse to navigate an operating system effectively, a data analyst does not need to understand the internal, low-level computer science mechanics of a Pandas DataFrame to extract highly valuable business insights. The overarching dream of the platform is to assist people in transitioning seamlessly into the new modern data world, empowering them to automate the manual jobs of the past through the use of Artificial Intelligence and advanced data methodologies.

To encapsulate this practical, mentorship-oriented teaching style into a highly scalable digital format, an advanced Retrieval-Augmented Generation (RAG) chatbot was engineered. This AI tutor was explicitly designed to mimic the pedagogical approach of a supportive data science instructor. It was built to move beyond answering generic coding questions by actively retrieving precise curriculum materials, guided projects, and methodologies taught directly within the DataSimple ecosystem.

The engineering of this platform required overcoming substantial hurdles in cloud architecture, data extraction pipelines, and advanced AI orchestration. The development evolved across four distinct phases: getting the learn function active to assist students directly with the lesson available on DataSimple to guided them

to the most valuable lesson (Phase 1); setting up the test function to evaluate where students' levels are at and guide them to appropriate projects (Phase 2); enabling an advanced debugger that collects the most recent documentation from the internet to help debug a student's code (Phase 3); and implementing a feedback loop where students can interactively offer suggestions for the future development of DataSimple.education (Phase 4). Ultimately, the seamless execution of these four phases will be enabled by an advanced multi-agent orchestration model, utilizing collaborative patterns with Amazon Bedrock Agents, Strands Agents, and Amazon Nova to dynamically route tasks.

This comprehensive research report details the exhaustive technical implementations, cost-optimization strategies, and multi-agent architectural patterns that bring the DataSimple AI Tutor to life.

Cloud Economics and Vector Database Selection

Building an intelligent generative AI application is merely the surface layer of modern machine learning engineering; constructing a scalable, highly automated, and economically viable data pipeline to feed that intelligence represents the true architectural challenge. For an educational platform like DataSimple, cost optimization is an important consideration dictating architectural choices. The explicit objective was to design a production-ready, highly scalable system that maintained a zero-cost standing footprint when not actively processing student queries.

When provisioning an AWS Bedrock Knowledge Base, the default "Quick Create" configuration automatically deploys an Amazon OpenSearch Serverless collection as the underlying vector database. While OpenSearch is a highly robust, enterprise-grade search engine capable of indexing terabytes of unstructured corporate data, it presents a massive financial trap for portfolio, educational, or highly optimized applications. OpenSearch Serverless bills for Minimum Compute Units (OCUs) simply to keep the database awake, indexed, and ready to answer potential queries. This baseline wakefulness costs approximately \$0.24 per hour. Consequently, an empty OpenSearch database incurs a standing cost of roughly \$150 to \$250 per month, totaling nearly \$2,400 annually, even if exactly zero web traffic hits the application and zero students ask a single question.

Alternative native relational databases, such as Amazon Aurora PostgreSQL equipped with the pgvector extension, were evaluated. However, establishing this infrastructure securely requires deploying a Virtual Private Cloud (VPC), configuring public and private subnets, establishing strict database security groups, managing master passwords, and manually executing SQL queries to generate the vector tables before Bedrock can even initiate a connection. Beyond the hours of complex

network engineering required, a baseline Aurora instance still incurs a steady monthly fee of \$40 to \$60 just to remain idle.

To circumvent these prohibitive cost structures, the architecture was fundamentally re-engineered to utilize Pinecone as the custom vector store backend.

Vector Database Solution	Infrastructure and Network Requirements	Monthly Standing Cost	Verdict for Educational RAG Workloads
Amazon OpenSearch Serverless	Automated via the AWS Bedrock Quick Create console.	~\$150 - \$250 (Driven by Minimum OCUs).	Extreme overkill; financially prohibitive for lightweight, low-traffic educational applications.
Amazon Aurora PostgreSQL (pgvector)	Requires extensive VPC setup, subnets, security groups, and manual SQL initialization.	~\$40 - \$60 (Idle instance compute time).	Demands complex network engineering and retains an unacceptable idle financial penalty.
Pinecone (Serverless Starter Tier)	API integration facilitated securely via AWS Secrets Manager. Native Bedrock support.	\$0.00 (Scales to true zero for up to 2GB of data).	The undisputed optimal choice. Lightning-fast, zero maintenance, and fully cost-optimized.

By securely locking the Pinecone API credentials within a digital vault using AWS Secrets Manager, Bedrock handles the computationally intensive tasks of chunking and embedding the educational Markdown files, while Pinecone acts as the silent, zero-cost storage layer. The integration requires the database metric to be set to Cosine similarity, and the vector dimensions must perfectly match the embedding model's output—specifically, 1024 dimensions when utilizing the highly cost-effective Amazon Titan Text Embeddings v2 model.

AWS Secrets Manager represents the only flat recurring charge in this highly optimized architecture, costing exactly \$0.40 per month to securely hold the API key. By swapping out OpenSearch for Pinecone, the entire data engineering pipeline—encompassing Amazon S3 storage, Lambda function invocations, API Gateway routing, and the Secrets Manager vault will fall within the AWS Free Tier.

The only variable costs incurred are tiny fractions of a cent for the Bedrock LLM invocation tokens during active, live chats with students.

Phase 1: Data Extraction, Organization, and the Environment Battle

The foundation of any high-performing Retrieval-Augmented Generation system is clean, semantically dense, and impeccably organized data. For the DataSimple platform, this required extracting deep pedagogical knowledge from nearly 100 Guided Projects, Data Tips, and YouTube video lessons. However, Phase 1 of the development, the mission to find, download, and organize these educational videos, turned into the biggest challenge of the first phase.

The Library Clash and the yt-dlp Pivot

The initial data extraction strategy attempted to utilize a new pre-release version of Python (version 3.14) alongside the standard youtube-transcript-api library. Because the Python architecture was so new, the original transcript library could not handle the updated internal structures, resulting in persistent, unresolvable `AttributeError` crashes. Furthermore, the local development terminal struggled to locate the correct Python executable due to overlapping, conflicting installations across Anaconda environments and the Windows Store.

To establish a stable data engineering environment, the system was downgraded to a highly stable Python 3.12 build. Because the youtube-transcript-api continued to throw ghost errors even in the stable environment, the architecture pivoted entirely to yt-dlp, an incredibly robust, industry-standard downloading library. While yt-dlp successfully and securely pinged YouTube to extract official video titles and subtitle files, it introduced a secondary, highly complex data engineering challenge: formatting.

Regex Data Scrubbing and the Code-Spacing Fix

The raw .vtt files downloaded by yt-dlp were explicitly designed for rendering text on video players, not for ingestion by Large Language Models. The raw files were heavily cluttered with useless metadata headers, hidden HTML tags (such as `<c>`), and continuous timestamp tokens (e.g., `00:00:13.000 --> 00:00:15.000`). Injecting this raw, unscrubbed formatting into an AWS Bedrock Vector Database would waste massive amounts of LLM context window space, severely degrading the AI's retrieval

performance and artificially inflating Bedrock token costs.

A custom Extract, Transform, Load (ETL) pipeline script was engineered to clean this data. The script utilized dynamic pathing via the `os.path.dirname` command to ensure clean output folders were systematically generated in the correct relative directory. More importantly, it deployed `re.sub` Regular Expression loop to iterate through the messy `.vtt` files, stripping away all timestamps and HTML tags to isolate the pure spoken words of the curriculum.

Resolving the Context Disconnect and Notebook Noise

Building the raw text transcript files was only the first step; organizing them for optimal vector search required solving a structural issue known in data engineering as the "Context Disconnect".

Initially, the curriculum repository consisted of standalone `.txt` transcript files and their corresponding `.ipynb` Jupyter Notebooks. If these files were uploaded to AWS Bedrock as separate entities, the AI would chunk and index the theoretical explanations entirely independently from the executable code. Consequently, when a student asked a complex data science question, the semantic search algorithm might retrieve the conceptual explanation but completely fail to retrieve the runnable code block that demonstrated it, leading to incomplete answers or severe AI hallucinations.

Furthermore, raw `.ipynb` files inherently have a lot of noise for them to be practical for LLM ingestion. They are heavily bloated with messy JSON formatting, line execution counts, and massive blocks of base64 encoded image data (such as embedded Matplotlib or Seaborn visualizations), creating a lot of noise that wouldn't be valuable in answering students' questions.

To rectify both the Context Disconnect and the noise-y Notebook data, a Python script was deployed using the `nbconvert` library. This script required strict naming conventions, demanding that matching notebooks and transcripts share the exact same base filename (e.g., `tip-sklearn-ard-regression.ipynb` and `tip-sklearn-ard-regression.txt`) before processing. The script parsed the notebook structure, stripped out the execution metadata and image data, transformed the remaining code into clean Markdown, and aggressively stitched it to the matching transcript to create a unified document (`.md` file).

This merge ensured that the Bedrock vector database chunking algorithm would process Brandyn's theoretical explanation immediately followed by the exact code that executed it, permanently and mathematically binding the context.

Ground Truth and the Exclusion of Templates

Regarding exactly what code to include in this newly formed vector database. For practice projects we frequently utilize fill-in-the-blank templates (e.g., `df.groupby('___').mean()`) to test student comprehension. However, including these incomplete templates in the RAG corpus causes severe AI hallucinations. An LLM might retrieve a blank template and output it directly to the user as a solution, or worse, it might attempt to blindly guess the missing syntax rather than providing the intended, official curriculum solution.

Consequently, the Pinecone database was strictly isolated as a "Ground Truth" repository. The pipeline was explicitly designed to embed only complete, 100% correct, runnable solution notebooks. To prevent the AI from acting as a simple code dictionary and giving away these solutions to struggling students without effort, the pedagogical restriction was offloaded entirely to the AWS Bedrock System Prompt. The Master Prompt instructed the AI to act as a dedicated tutor, utilizing the retrieved ground-truth projects to understand the context, but requiring it to guide the student to the answer using iterative hints.

Metadata Flatness and the AWS Ingestion Engine

To navigate the 14MB of course data efficiently, the architecture relied on metadata sidecar files (`.metadata.json`) created from a master curriculum metadata JSON. In the world of vector databases, metadata acts as a highly efficient, programmatic filtering system. Instead of forcing the AI to mathematically scan every single vector in the database, Bedrock can use metadata to instantly filter chunks by specific categories like `difficultyLevel`, `topic`, and `contentType`, vastly improving retrieval accuracy while reducing computational costs.

However, the automated Python pipeline initially produced rich JSON structures containing nested lists and arrays (e.g., `"tags": ["machine learning", "python", "seaborn"]`). The AWS Bedrock ingestion engine is exceptionally strict regarding metadata formatting. It requires a specific JSON wrapper called `metadataAttributes` and requires that all values within that wrapper be strictly flattened strings, numbers, or booleans. Because the engine detected arrays instead of flat strings, it completely rejected the nested JSON files, crashing the entire ingestion job.

To resolve this without manually editing hundreds of lines in the master metadata JSON, a specialized Python script was developed. The script looped through the local directory, utilized the `isinstance` function to detect any arrays or nested dictionaries, and dynamically flattened them into comma-separated strings (e.g., `"tags": "python, data visualization, seaborn"`) before syncing the corrected payload

directly to the AWS S3 Clean bucket.

Building the Serverless Frontend: API Gateway and Wix Velo

To deliver this educational intelligence to the student, a decoupled, highly secure serverless frontend architecture was established. Rather than hardcoding secure AWS credentials directly into the client-side website, a severe cybersecurity vulnerability, the system utilized an Amazon API Gateway as the public-facing "door" and an AWS Lambda function as the secure "courier".

The frontend interface was constructed using Velo by Wix, a powerful JavaScript environment that allows for custom web application development integrated directly within a visual builder. A critical user interface decision was made to build a dedicated AI Tutor page rather than a ubiquitous floating hover widget. Data science answers frequently include large blocks of Python code, tabular data, and complex step-by-step logic. A dedicated, full-screen page maximizes screen real estate, ensuring that these rich outputs format beautifully without the cramped, unreadable constraints of a 300-pixel-wide popup box.

The Velo JavaScript was programmed to capture the user's text input, format it into a standardized JSON payload, and execute an asynchronous HTTP POST request directly to the API Gateway endpoint. Crucially, the API Gateway required strict Cross-Origin Resource Sharing (CORS) configurations. Browsers utilize CORS as a fundamental security feature to prevent unauthorized domains from making backend API calls. Without explicitly defining Access-Control-Allow-Origin as * (or the specific domain) and mapping the mandatory OPTIONS preflight requests in the API Gateway console, modern web browsers aggressively block the Wix website from communicating with the AWS backend, resulting in silent network failures.

The Lambda function, operating on a Python runtime and utilizing the boto3 SDK, received the payload and executed the `retrieve_and_generate` API to query the Bedrock Knowledge Base. During initial testing, an architectural limitation was discovered: the 3-Second Rule. By default, AWS Lambda functions are configured with a strict 3-second timeout limit to prevent runaway compute costs. However, an advanced model like Claude 3.5 Sonnet requires roughly 5 to 10 seconds to read a student's prompt, mathematically search the Pinecone vector database, synthesize the curriculum materials, and generate a highly contextual educational response.² Because the function was consistently dying at the 3-second mark, it never returned the payload to Wix, resulting in continuous 500 Internal Server Errors. The architecture was optimized by increasing the Lambda timeout configuration to 30 seconds, ensuring the generative AI had sufficient execution time.

Phase 2: The Test Function and Project Guidance

While answering direct, one-off queries is the baseline functionality of a simple RAG system, true educational utility requires dynamic state management to facilitate interactive practice tests and project routing. Phase 2 of the platform's development involved setting up a robust testing function to evaluate student proficiency and providing strict architectural boundaries for project recommendations.

The Stateless Goldfish and Session Memory

By design, standard AWS Lambda functions are entirely stateless. Every time a student clicked "Send" on the Wix interface, the API Gateway invoked a brand new, completely isolated execution environment. If the AI asked a multiple-choice practice question and the student replied with the single letter "a", the stateless function received the letter "a" in a total vacuum. Lacking any historical conversation context, Bedrock would blindly search the Pinecone database for the mathematical vector of the letter "a", find zero relevant curriculum matches, and trigger its hardcoded out-of-bounds error.

To simulate long-term conversational memory, the architecture implemented Bedrock's native sessionId tracking. The Python backend was updated to extract the unique sessionId generated by AWS and return it to the Wix frontend within the response payload. The Velo JavaScript was heavily modified to declare a global currentSessionId variable, capture the memory token from the AWS response, and attach it to all subsequent POST requests. This continuous passing of the memory tag allowed the AI to recall the entire chat history natively, contextualizing short answers like "a" or "b" within the exact framework of the ongoing practice test.

The Context Overwrite Problem

Even with session memory successfully enabled, the system encountered a "Context Overwrite" problem, a fundamental vulnerability in linear RetrieveAndGenerate pipelines. When a student submitted an answer (e.g., "My answer to the practice test is: a"), the retrieve_and_generate API fundamentally treated that incoming sentence as a new semantic search query. It queried the Pinecone database for the contextual words "practice test" and "answer".

Pinecone, performing its mathematical retrieval perfectly, would return fragments of *other* practice tests found elsewhere in the curriculum data. Claude, operating

under strict instructions to prioritize retrieved search context, would read the newly fetched questions, become immediately distracted, and completely forget that it was supposed to be in a "Grading Mode". It would essentially overwrite its own conversational memory with the fresh search results, often outputting a completely different question rather than grading the student's submission. In one documented instance of this distraction loop, the AI graded the exact same question twice, argued with itself, and provided two entirely different "correct" answers to the user.

Semantic Camouflage and Padding Tricks

To force the AI into the correct behavioral state, a complex series of frontend context injections were engineered within the Wix Velo code. The JavaScript was updated to monitor the length of user inputs; if it detected a short, 1-to-2 character answer, it invisibly appended highly specific keywords to the user's message before transmitting the payload to AWS.

Initially, the script appended direct instructions like, "Please enter Grading Mode. Grade my answer, tell me if I am right or wrong, and provide the full explanation". However, this triggered an aggressive AWS Bedrock Security Guardrail. The AWS security layer constantly scans incoming prompts for hackers attempting to "jailbreak" or override the AI's core instructions—a cyberattack known as Prompt Injection. Because the security system detected a user attempting to issue strict formatting commands, it panicked, assumed a breach, and slammed the door, outputting the hardcoded response: "Sorry, I am unable to assist you with this request".

The architecture immediately pivoted to an Innocent Padding. Instead of issuing commands, the frontend simply provided deep, purely descriptive semantic context: "My answer to the practice test question is: [a]". This entirely bypassed the Prompt Injection security guardrails while providing enough keyword density to prevent an "Empty Search Crash"—a hardcoded AWS error where returning zero vector results causes the entire API to panic and fail before the AI can even read the chat history.

Project Suggestions and Hallucination Control

Once the student's level is evaluated through testing, a critical pedagogical component is project-based, experiential learning. Designing an AI to provide intelligent project suggestions based on those test results presents unique engineering challenges regarding generative hallucination, scope creep, and formatting standardization.

If prompted poorly, a powerful LLM might recommend generic, off-platform projects (e.g., "Go to Kaggle and download the Titanic dataset") or invent entirely non-existent curriculum links. To exert absolute control over the AI's behavior and ensure it acted purely as an extension of the DataSimple educational ecosystem, a highly engineered MASTER_PROMPT was integrated into the generative configuration.

The "Two Projects" Rule and Tone Alignment

The prompt architecture enforced strict behavioral and tonal constraints. First, it demanded a warm, enthusiastic, and highly encouraging tone, explicitly commanding the AI to speak directly to the student and validate their curiosity, perfectly mirroring the human teaching style of the platform's creator. The AI was explicitly forbidden from utilizing robotic phrasing or exposing its internal mechanics, such as outputting "The provided context shows...".

To meticulously manage project suggestions, the "Two Projects" rule was hardcoded into the master prompt logic. Whenever a student inquired about project ideas, next steps, or practical applications, the AI was mathematically constrained to provide exactly two distinct project suggestions retrieved strictly from the Pinecone vector database. Furthermore, the AI was required to briefly explain the fundamental pedagogical reasoning behind *why* each project was a perfect fit for the specific concept the student was currently learning, reinforcing the diagnostic and mentorship-oriented style of the platform.

Eradicating Generative Hallucinations

To ensure the AI strictly recommended official, accessible content, aggressive hallucination guardrails were established within the logic. Rule 8 of the Master Prompt explicitly prohibited the recommendation of external platforms, generic documentations, or assumed URLs. The AI was commanded to output only the exact lesson titles explicitly mentioned in the search results.

To facilitate this strict matching, a crucial Data Engineering update was applied to the raw .md files before they were pushed to the S3 clean bucket for vectorization. A strict metadata block was injected at the very top of every single curriculum file (e.g., # Official Lesson Title: [Exact Name]). This structural update allowed the Librarian Agent to extract the precise, unalterable nomenclature of a "Guided Project" or "Data Tip" and serve it directly to the user. This ensured the student could easily locate the corresponding video or notebook on the DataSimple platform without encountering dead links, hallucinated URLs, or off-site distractions.

Finally, the prompt included an explicit "Out of Bounds" protocol. If a student asked

a question entirely unrelated to the embedded curriculum, the AI gracefully admitted the limitation, stating that the topic was not covered yet, and seamlessly redirected focus back to the existing materials. This protocol maintained the absolute integrity of the instructional environment, ensuring the AI remained a focused tutor rather than an unstructured, general-purpose chatbot.

The Ceiling of a Monolithic Pipeline

While these frontend workarounds—semantic camouflage, hidden context injection, and hyper-strict master prompting—successfully stabilized the initial prototype, they ultimately exposed the hard limitations of a monolithic, single-agent architecture. Forcing one Lambda function and a single LLM configuration to simultaneously act as a document retriever, a state-tracking test examiner, and a general tutor created a highly fragile system. Every new feature or conversational edge-case required increasingly complex string manipulation in the Wix frontend just to prevent the model from collapsing under its own context weight or getting distracted by unnecessary database searches.

The Paradigm Shift: Multi-Agent Orchestration (Planned)

To achieve true enterprise scalability and permanently eradicate the "Context Overwrite" problem, the system architecture required a fundamental redesign. The solution was a pivot from a linear **RetrieveAndGenerate** API to a **Multi-Agent Orchestration** model utilizing Amazon Bedrock Agents.

By decoupling these diverse responsibilities into specialized, isolated AI entities, the system gains the ability to natively manage conversational memory, selectively bypass database retrieval when it is not needed, and dynamically route tasks based on the student's explicit intent.

The following section outlines the planned Multi-Agent architecture, coordinated by a central "Principal Router" that intelligently delegates tasks to three specialized sub-agents:

- **Learn:** A retrieval-specialized agent explicitly bound to the AWS Knowledge Base. It leverages the newly engineered JSON metadata sidecar files (hard database filters) to deliver precise curriculum pathways and project

recommendations without "guessing" document relevance.

- **Test:** A state-tracking agent operating completely independently of the Knowledge Base. By intentionally bypassing document retrieval, it eliminates token snowballing and "distracted retriever" hallucinations. This allows it to natively remember the multiple-choice options it just generated, grade the student's answer, and loop through practice tests without losing conversational context.
- **Debug:** An externally-facing agent equipped with API action groups (tools) to search the live internet. Rather than relying on static `.md` files, the Debugger can read current, live documentation (e.g., the latest Scikit-Learn or Pandas versions) to troubleshoot broken student code without hallucinating deprecated syntax.
- **Feedback:** An interactive agent dedicated to collecting and processing student suggestions. Rather than routing input through standard chat responses, this agent systematically captures user feedback for Teacher Brandyn or for the future development of DataSimple.education, securely logging it into a serverless NoSQL database to inform continuous platform improvement.
- **Get to Know Brandyn:** This section is a dedicated space for the DataSimple community to follow along with behind-the-scenes updates and ongoing projects. It's a way to share transparent progress on current endeavors—like AWS certifications, the Nativox app, and writing Parallel Productivity—and keep growing together.

Phase 3: Live Code Debugging via Web Search (Planned)

To realize the multi-agent vision outlined above and permanently resolve the limitations of the historical knowledge base, Phase 3 is planned to fully implement the **Amazon Bedrock Multi-Agent Collaboration** pattern. This future framework will replace the single, stressed Lambda function with the hierarchical state machine of specialized AI agents.

Architectural Pattern	Core Orchestration Mechanism	Memory & Context Handling	Primary Limitation or Tradeoff
Monolithic RAG (Lambda)	Single function executing linear	Requires manual passing of	Highly susceptible to Context Overwrite;

	RetrieveAndGenerate for every prompt.	sessionId tokens via frontend API payloads.	cannot bypass the vector search requirement for simple conversational turns.
Agentic AI Orchestration (Planned)	Central Supervisor Agent dynamically routes tasks to specialized domain Sub-Agents.	Native context retention and seamless session management across multiple autonomous interactions.	Requires high initial configuration complexity, strict role demarcation, and complex schema definitions.

When fully developed, the Supervisor Agent will act as the principal orchestrator, dynamically routing the student's unstructured query to the Librarian, Examiner, or Debugger at runtime.

The most advanced and dynamic node in this planned collaborative graph will be the Debugger Agent. Because it will be purposefully decoupled from the historical Pinecone database, it will be equipped with Action Groups linked to live web search APIs, such as the Serper API for Google searches or Tavily AI for research-heavy inquiries.¹ Tavily can be deployed seamlessly as a managed MCP (Model Context Protocol) server on Amazon Bedrock AgentCore, providing secure real-time web search and content extraction capabilities. When a student requests help debugging a trace error, the Debugger Agent will invoke a secure Lambda function to retrieve API credentials from AWS Secrets Manager and ping the live internet for the latest Python library documentation. Utilizing Tavily's "include answer" functionality, the system will generate a coherent LLM-powered summary of the findings, allowing the agent to formulate a real-time, code-accurate solution to fix the student's script directly in the chat.

Phase 4: Interactive Student Feedback Loop (Planned)

The final planned phase of the DataSimple AI Tutor architecture will focus on continuous improvement through student input. To implement a dynamic feedback

loop where students can interactively offer suggestions for the future development of the platform, a fully serverless architecture pattern will be established utilizing Amazon API Gateway, AWS Lambda, and Amazon DynamoDB.

When a student submits an idea, suggestion, or curriculum request via the Wix frontend interface, the payload will be transmitted as an asynchronous HTTP POST request directly to the API Gateway. The gateway will efficiently route this request to a dedicated AWS Lambda function acting as the backend processor. This Lambda function will parse the student's incoming text and securely write the data into a NoSQL Amazon DynamoDB table (e.g., a feedbacks table) designed specifically to hold user submissions.

By utilizing DynamoDB as the digital ledger for these suggestions, the platform will ensure rapid data insertion and high scalability. This specific architectural pattern will be highly cost-efficient; because it is entirely serverless, it will incur absolutely zero standing server management costs and bill only for the actual compute execution time and database storage used when a student actively submits a suggestion.

Works cited

Integrate dynamic web content in your generative AI application using a web search API and Amazon Bedrock Agents | Artificial Intelligence, accessed March 28, 2026, <https://aws.amazon.com/blogs/machine-learning/integrate-dynamic-web-content-in-your-generative-ai-application-using-a-web-search-api-and-amazon-bedrock-agents/>

Retrieving information from data sources using Amazon Bedrock Knowledge Bases, accessed March 28, 2026,

<https://docs.aws.amazon.com/bedrock/latest/userguide/kb-how-retrieval.html>

Guidance for Multi-Agent Orchestration on AWS, accessed March 28, 2026,

<https://aws.amazon.com/solutions/guidance/multi-agent-orchestration-on-aws/>

How Amazon Bedrock Agents works, accessed March 28, 2026,

<https://docs.aws.amazon.com/bedrock/latest/userguide/agents-how.html>